

Production Supporting Systems in Factories

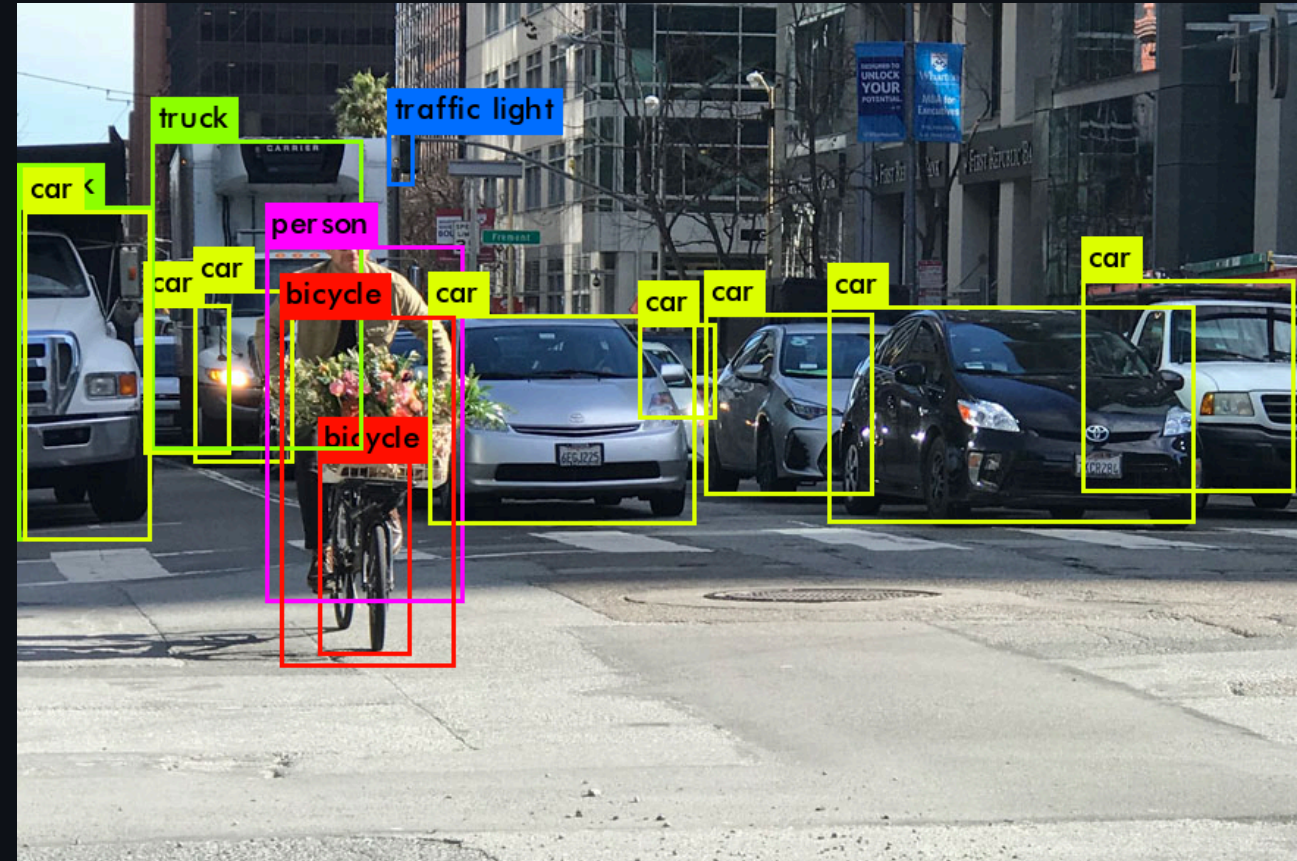
ระบบสนับสนุนการผลิตในโรงงานอุตสาหกรรม

Machine Learning

| Object Detection

Object Detection

- Car
 - Top: 500, Bottom: 200, Left: 50, right: 400
 - 50%
- Bicycle
 - ...
 - ...



COCO Dataset

- *Common Objects in Context*
- Large-scale image recognition dataset for object detection, segmentation, and captioning tasks.
 - Contains over 330,000 images.
 - Annotated with 80 object categories.
- <https://cocodataset.org/#explore>

DETR Model

- DEtection TRansformer (DETR) model by Facebook AI.
- Uses transformer architecture for object detection.
- Pre-trained on COCO dataset.
- Classes

Setting Up ML Server

- Get [code](#)
- `pnpm install`
- `pnpm run build`
- `pnpm run start`
- Test if server is running.
 - <http://localhost:3004>

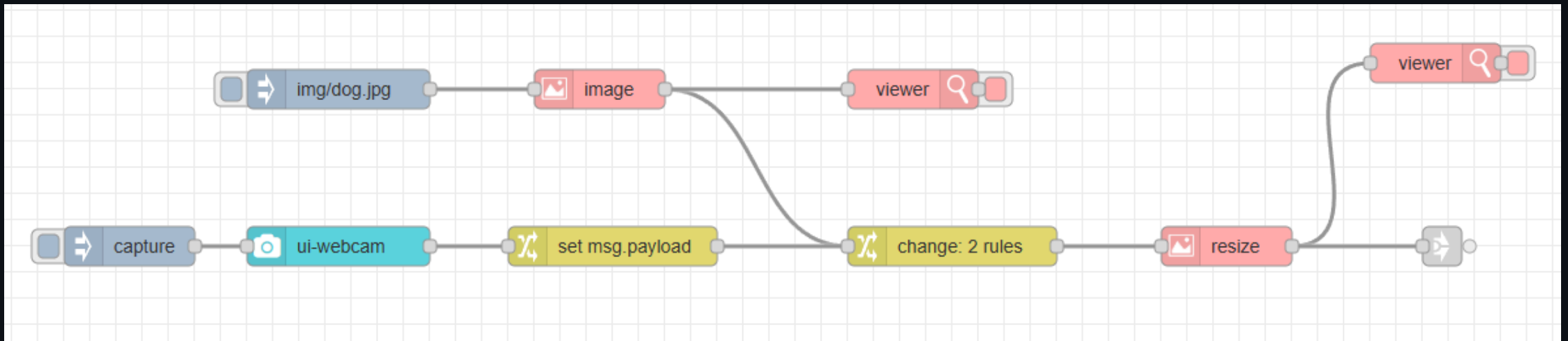
Install Additional Modules

- Go to Menu (top right) -> Manage palette -> Install
- Install the following modules:
 - `node-red-contrib-image-tools`
 - `@sumit_shinde_84/node-red-dashboard-2-ui-webcam`

Prepare Mock Images

- Copy image from `obj-detection-express-main/temp/img` to your node-red installation directory under `img` folder.

Image Input Flow



Inject Node Configuration

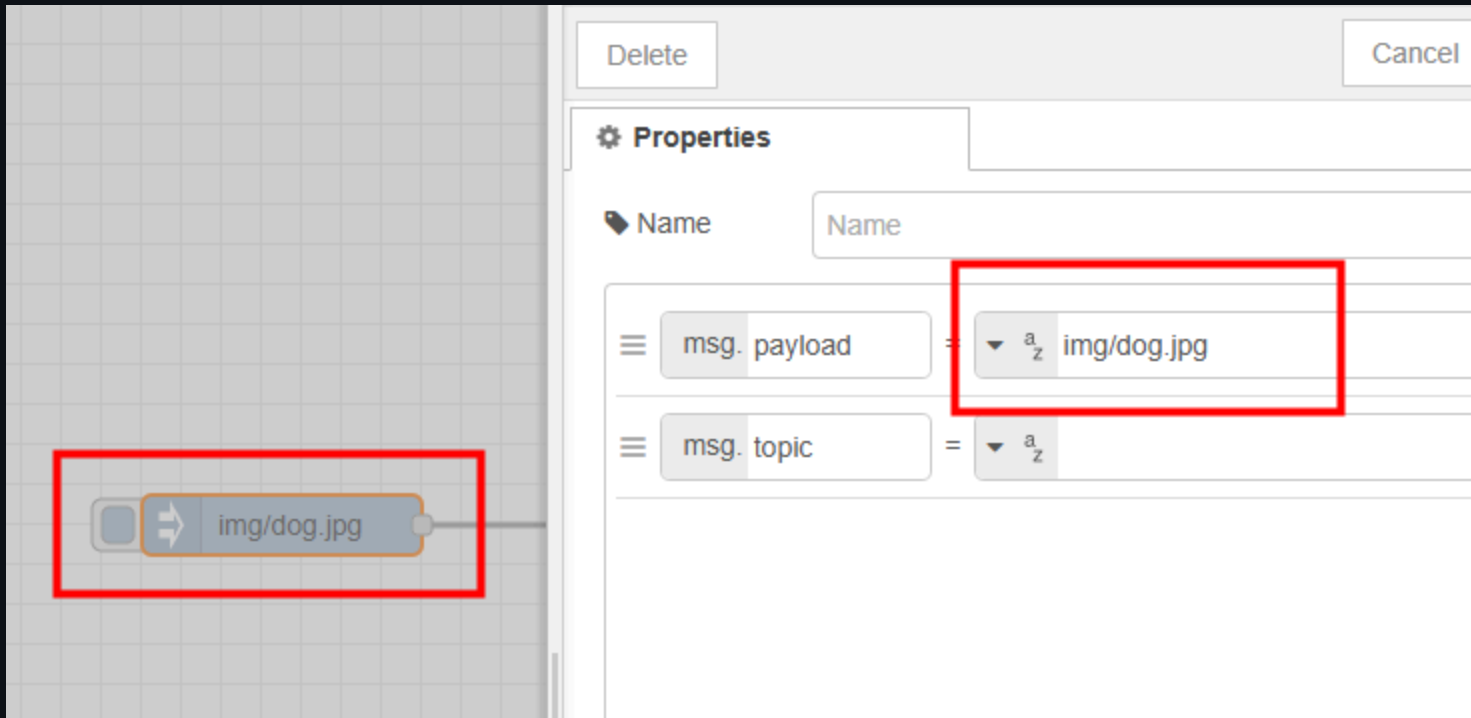
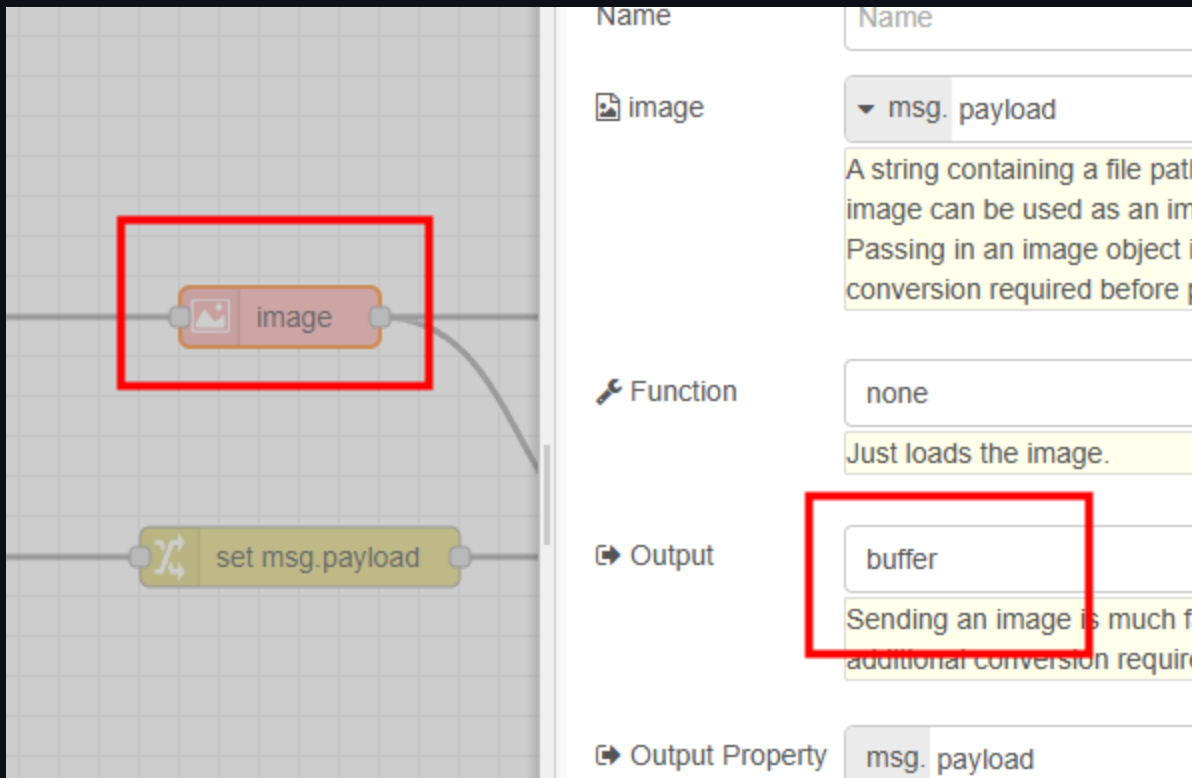


Image Node Configuration



The screenshot displays the Node-RED web interface. On the left, a workflow is visible on a grid background. It consists of two nodes: an 'image' node (orange with a picture icon) and a 'set msg.payload' node (yellow with a double-headed arrow icon). The 'image' node is highlighted with a red rectangular box. A wire connects the output of the 'image' node to the input of the 'set msg.payload' node. On the right, the configuration panel for the selected 'image' node is shown. It has a white background and a light gray border. The 'Name' field at the top contains 'image'. Below it, the 'Function' field is set to 'none'. The 'Output' field is set to 'buffer', and this field is highlighted with a red rectangular box. The 'Output Property' field at the bottom is set to 'msg. payload'. The configuration panel also contains descriptive text: 'A string containing a file path... image can be used as an im... Passing in an image object... conversion required before p...' and 'Just loads the image.'

Field	Value
Name	image
Function	none
Output	buffer
Output Property	msg. payload

Change Node Configuration

The screenshot displays the Node-RED web interface. On the left, a workflow is visible with an 'image' node connected to a 'set msg.payload' node, which is highlighted with a red rectangle. Below it, an 'http request' node is connected to a 'json' node. On the right, the 'Properties' panel for the 'set msg.payload' node is shown. It features a 'Name' field and a 'Rules' section. The 'Rules' section contains two rule entries, both highlighted with red rectangles:

- Rule 1: Action 'Set', Target 'msg. width', Value '300'.
- Rule 2: Action 'Set', Target 'msg. height', Value '-1'.

Image Node Configuration (Resize)

The screenshot displays the Node-RED web interface. On the left, a workflow is visible with a 'resize' node (orange box with a red border) connected to a 'viewer' node. Below this, there are other nodes like 'mat Prediction Data', 'mat Count Data', 'mat Select-Class Data', and 'Show Image'. On the right, the configuration panel for the 'resize' node is shown. It includes a 'Function' dropdown set to 'resize', an 'Output' dropdown set to 'buffer', and 'Output Property' set to 'msg. payload'. Below these, there are two dropdown menus for 'w' (width) and 'h' (height), both set to 'msg. width' and 'msg. height' respectively. Red boxes highlight the 'resize' node in the workflow and the 'Function', 'Output', 'w', and 'h' configuration options in the panel.

Function: **resize**

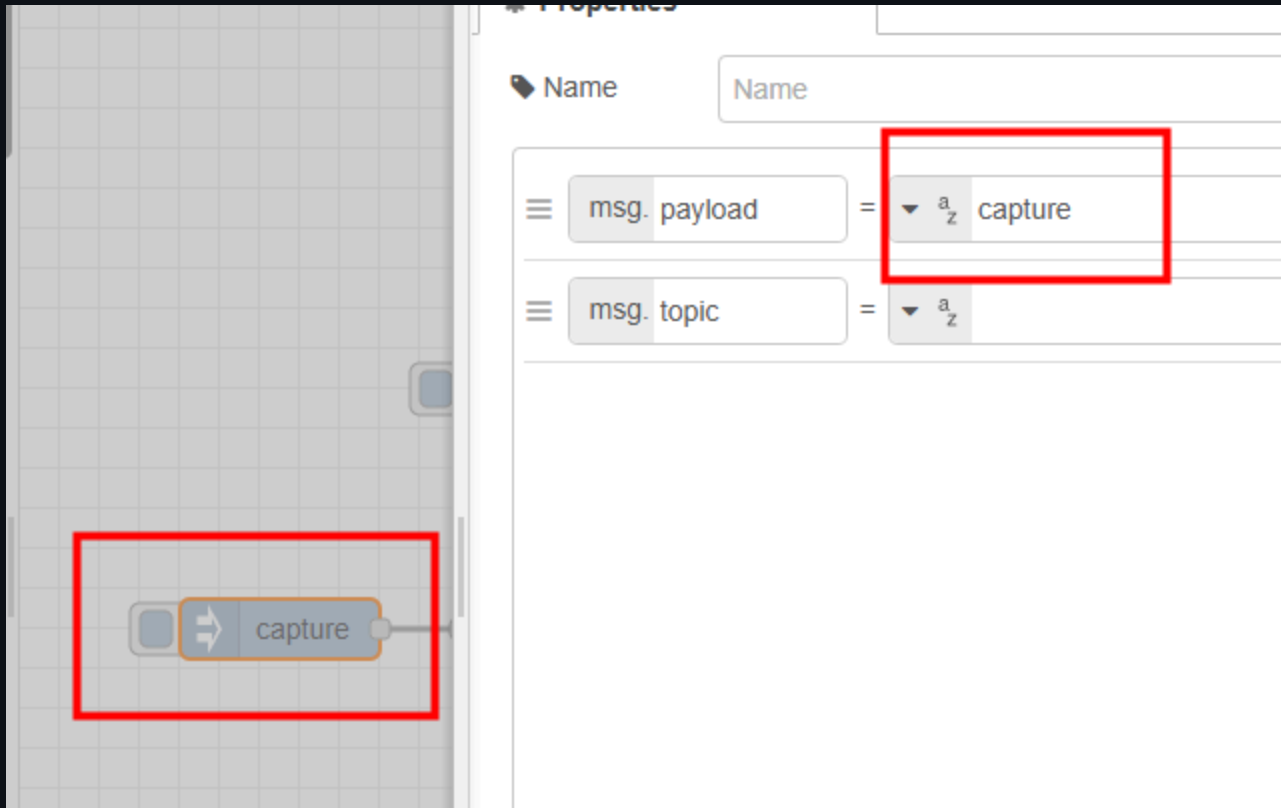
Output: **buffer**

Output Property: **msg. payload**

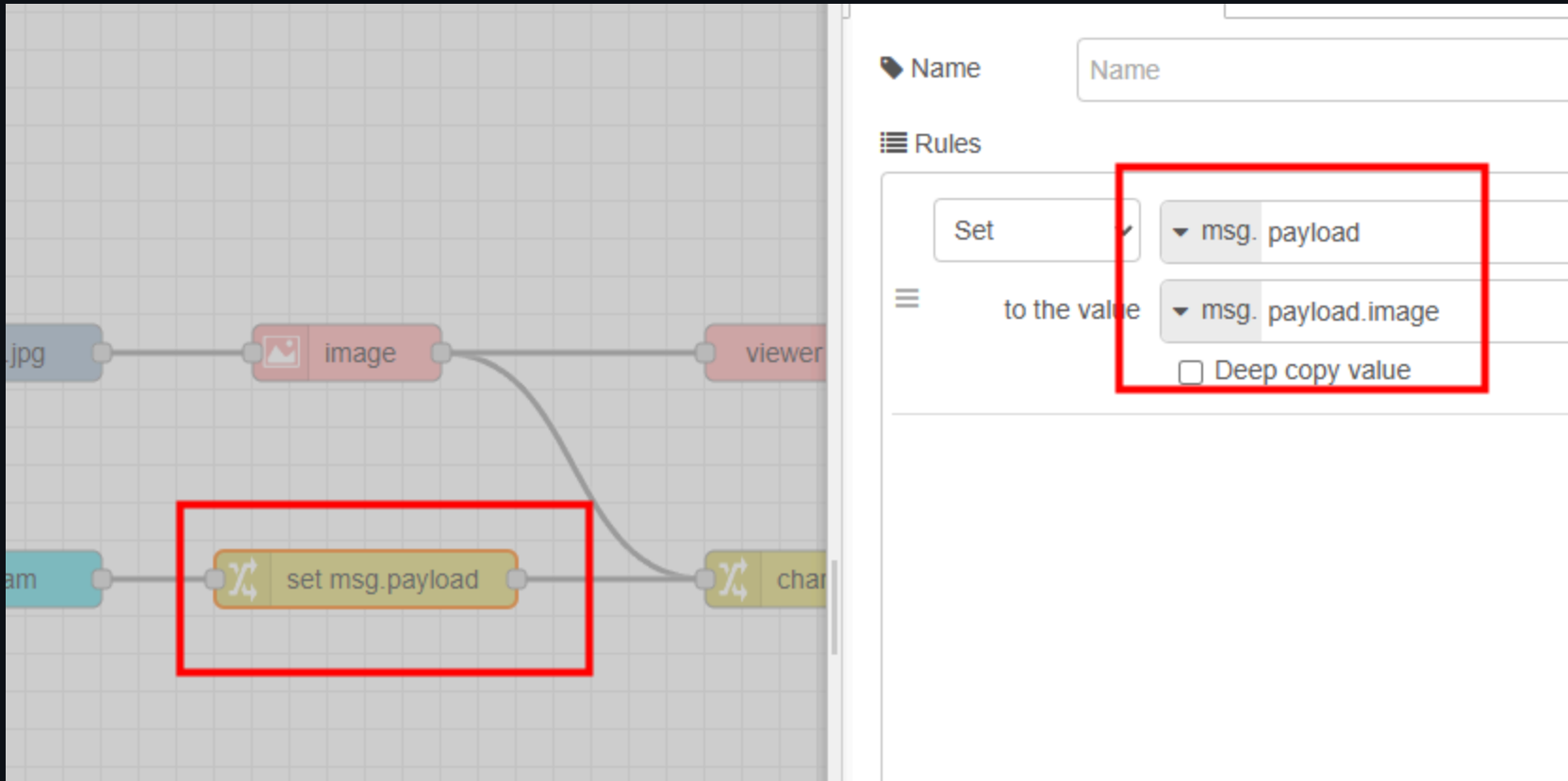
</> w: **msg. width**

</> h: **msg. height**

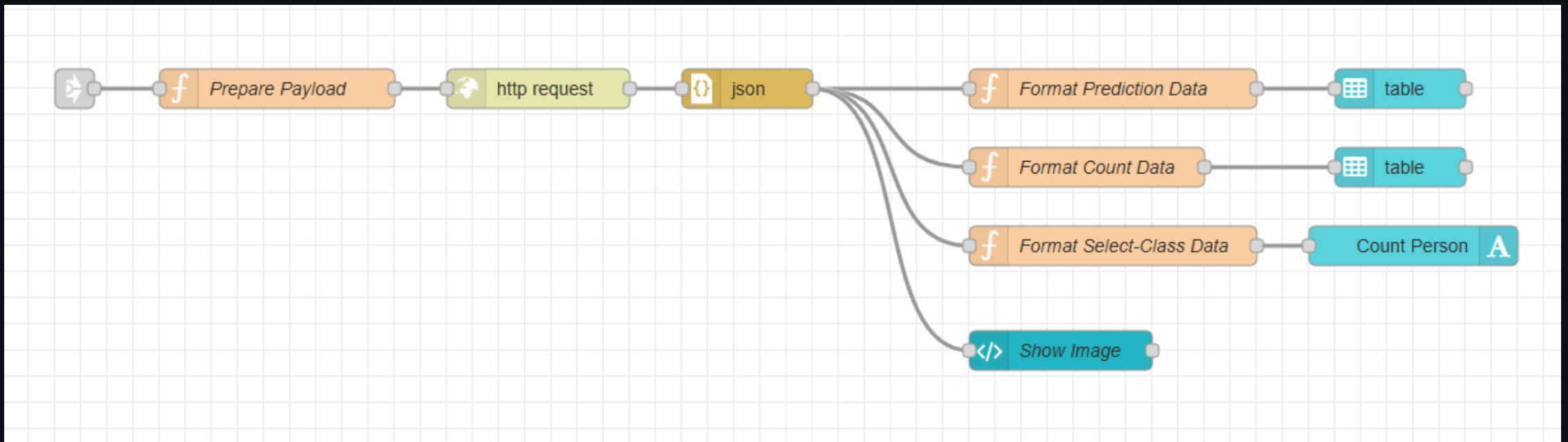
Inject Node Configuration (Capture)



Change Node Configuration (Webcam)



Object Detection Flow



Prepare Payload Function Node

```
msg.headers = {  
  "content-type": "multipart/form-data",  
};  
msg.payload = {  
  img: {  
    value: msg.payload,  
    options: {  
      filename: "a.jpg",  
      contentType: "image/jpeg",  
    },  
  },  
};  
msg.url = "http://localhost:3004/upload"; // ML server endpoint  
return msg;
```

HTTP Request Node Configuration

The screenshot displays the Node-RED web interface. On the left, a workflow is visible on a grid background. It includes a blue 'http in' node, a red 'image' node, a red 'viewer' node, a cyan 'am' node, a yellow 'set msg.payload' node, a yellow 'change' node, an orange 'http request' node (highlighted with a red rectangle), and a yellow 'json' node. On the right, the 'Properties' sidebar is open for the 'http request' node. It features a red rectangle around the 'Method' dropdown, which is set to 'POST'. Below this, the 'URL' field contains 'https://'. A list of checkboxes follows: 'Enable secure (SSL/TLS) connection', 'Use authentication', 'Enable connection keep-alive', 'Use proxy', 'Only send non-2xx responses to Catch node', and 'Disable strict HTTP parsing'. At the bottom, the 'Return' dropdown is set to 'a UTF-8 string', and the 'Headers' section is partially visible.

Format Prediction Data Function Node

```
const predictions = msg.payload?.predictions;
if (!Array.isArray(predictions)) {
  msg.payload = [];
  return msg;
}

const dt = new Date();
const datestring = dt.toLocaleDateString();
const timestring = dt.toLocaleTimeString();
msg.payload = predictions.map((d) => ({
  Class: d["label"],
  Score: `${Math.round(d["score"] * 100)}%`,
  Location: JSON.stringify(d.box),
  Time: `📅 ${datestring} ⌚ ${timestring}`,
})));
return msg;
```

Table Node Configuration

The screenshot displays the configuration interface for a 'Table' node. On the left, a workflow diagram shows a 'Data' node connected to a 'table' node, which is highlighted with a red rectangle. Below it, another 'table' node is connected to a 'Count Person' node. On the right, the configuration panel for the selected 'table' node is shown. It includes the following settings:

- Size:** auto
- Label:** optional label
- Class:** Optional CSS class name(s)
- Max Rows:** 0
- Action:** Replace (highlighted with a red rectangle)
- Breakpoint:** defaults: Mobile (< 576px)
- Interaction:** None
- Search:** ☒ Show

Format Count Data Function Node

```
const countsArr = msg.payload?.countsArr;
if (!Array.isArray(countsArr)) {
  msg.payload = [];
  return msg;
}

const dt = new Date();
const datestring = dt.toLocaleDateString();
const timestring = dt.toLocaleTimeString();
msg.payload = countsArr.map((d) => ({
  Class: d["class"],
  Count: d["count"],
  Time: `📅 ${datestring} ⌚ ${timestring}`,
})));
return msg;
```

Format Select-Class Data Function Node

```
// Select class here
const selectClass = "person";

// Check
const countsObj = msg.payload?.countsObj;
if (typeof countsObj !== "object" || countsObj === null) {
  msg.payload = 0;
  return msg;
}

// Code
msg.payload = countsObj?.[selectClass] ?? 0;
return msg;
```

Template Node Configuration

```
<template>
  <div>
    
  </div>
</template>

<script>
  export default {};
</script>
<style></style>
```